Learning to Play Tetris with World Models

Andrei Jardan Mengfan Wu

Abstract

Joint-embedding predictive architectures have made latent world models a practical basis for planning, but they are almost exclusively evaluated on sparse environments in which a few objects move against a static background. We show that current models fail on dense 2D dynamics: on Tetris, a configurable environment we introduce as a world model benchmark, LeWM [16] plans successfully in only 7% of episodes. We trace this failure to how ViT tokens enter the latent state, and propose the patch projector*, a cheap and simple architectural change, which raises planning success rate to 92%, at unchanged rollout cost. We further use the resulting model to play Tetris directly from a sparse row-clearing objective, a setting prior latent planning work does not address.*

1. Introduction

A world model that predicts the consequences of actions allows an agent to plan by searching over imagined futures rather than acting in the environment [8, 9]. Joint-embedding predictive architectures (JEPAs) [14] make this attractive at scale: by predicting in a learned latent space instead of pixel space, they avoid spending capacity on visually salient but behaviourally irrelevant detail. Recent latent world models built on this principle plan successfully in continuous control environments [16, 20, 21].

These successes share a common setting. Benchmarks such as PushT, Reacher and TwoRoom present a small number of objects moving against a static background, so a latent state that captures a handful of object positions is sufficient. Many environments of interest are not like this. We study Tetris, which is dense: the occupied fraction of the board grows throughout an episode, discrete, and non-local, since a single action can clear a line and alter the state at many distant positions simultaneously. Existing latent world models degrade sharply here: LeWM [16] plans successfully in only 7% of episodes.

We argue that this failure is not a matter of capacity or training data but of how the encoder's ViT tokens are aggregated into a latent state. To alleviate this problem, we introduce the *patch projector*, a single linear layer shared across all patches that reduces each patch token to a small dimension before it is concatenated with the class token.

Finally, we note that latent planning is almost always evaluated as goal-state reaching: given a target observation, recover an action sequence that arrives at it. Real objectives are rarely available as target images. Tetris admits a natural abstract goal: clear as many rows as possible, and we show that a linear scorer over the learned latent, optimized with CMA-ES [12] against this sparse reward, is sufficient to turn the world model into a Tetris player.

Our contributions are:

- The **patch projector**, an architectural change that interpolates between class-token-only and stacked-patch-token latents, improving planning success on Tetris by an order of magnitude at no additional rollout cost (Tab. 1).
- A configurable **Tetris environment** as a world model benchmark with dense, discrete and non-local dynamics, on which current methods either fail or are prohibitively slow.
- An **abstract-goal planning** setup that plans towards a sparse reward rather than a goal observation.

2. Related Work

Joint-embedding predictive architectures. JEPAs [14] learn representations by predicting the embedding of one view of an input from another, rather than reconstructing it, and have been applied to images [1] and video [4]. Because the objective is minimized by constant embeddings, they require an explicit mechanism against representation collapse: variance-covariance regularisation [3], or the sketched isotropic Gaussian regulariser (SIGReg) of LeJEPA [2], which matches the latent distribution to an isotropic Gaussian along random one-dimensional projections. We adopt SIGReg and revisit its weighting in Sec. 4.4.

World models for planning. Reconstruction-based world models learn dynamics in a latent space trained to predict pixels [10, 11, 17], which ties model capacity to visual detail irrespective of its relevance for control. Latent world models avoid this by predicting embeddings directly: DINO-WM [21] plans over frozen DINOv2 [18] features, PLDM-WM [20] learns dynamics from reward-free offline data, and LeWM [16] trains encoder and predictor jointly end-to-end

from pixels. Games have long served as benchmarks for learned dynamics [6, 17], but existing latent planning evaluations remain confined to sparse continuous-control tasks, and to goal-state reaching rather than abstract objectives. We evaluate in a dense discrete environment and plan against a sparse reward.

Token compression in vision transformers. Reducing the *number* of ViT tokens is well studied: token merging [5] and pruning [19] discard redundant tokens, Perceiver-style resamplers [13] attend a fixed set of latents over the input, and register tokens [7] add capacity outside the patch grid. All preserve per-token dimensionality. We instead retain every patch, and thus the correspondence between tokens and board regions, while compressing along the channel dimension.

3. Methodology

3.1. The Tetris environment

We implement a configurable Tetris environment exposing the grid size, the subset of tetrominoes in play, and the length and visibility of the upcoming-piece queue. The action space is discrete and consists of six actions: `MOVE_LEFT`, `MOVE_RIGHT`, `MOVE_DOWN`, `HARD_DROP`, `ROTATE_LEFT` and `ROTATE_RIGHT`. Observations are RGB renderings of the board together with the piece queue at a resolution of 126×126. Unless stated otherwise, all Tetris experiments use a 8×12 grid and the reduced tetromino set $\{I, O, L, J\}$; Sec. 5 discusses the full-piece setting.

3.2. World model formulation

A world model comprises an encoder enc_θ mapping an observation $\mathbf{im}_t$ to a latent state $\mathbf{z}_t = \mathrm{enc}_\theta(\mathbf{im}_t) \in \mathbb{R}^{D_{\text{emb}}}$, and a predictor pred_ϕ that advances the latent state conditioned on an action, $\hat{\mathbf{z}}_{t+1} = \mathrm{pred}_\phi(\mathbf{z}_t, \mathbf{a}_t)$. Both are ViTs; we use a ViT-Tiny encoder, trained from scratch, with a patch size of 14 giving $N = 81$ patch tokens per observation. The action is embedded by a learned MLP, and injected into the predictor ViT through AdaLN-Zero conditioning.

Following LeWM [16], we train both networks jointly by minimizing

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \, \mathcal{L}_{\text{SIGReg}}, \tag{1}$$

where the prediction term is

$$\mathcal{L}_{\text{pred}} = \frac{1}{K} \sum_{i=1}^{K} \|\mathbf{z}_i - \mathrm{enc}_\theta(\mathbf{im}_i)\|_2^2 \tag{2}$$

$\mathcal{L}_{\text{SIGReg}}$ is the sketched isotropic Gaussian regularizer of LeJEPA [2], which prevents representation collapse by pushing the latent distribution towards an isotropic Gaussian along randomly sampled one-dimensional projections. The weight λ balances predictability against latent diversity.

3.3. Latent Grid Probe

Despite being the primary optimization objective, prediction loss is not a reliable indicator of the performance that a model will achieve in downstream tasks such as latent planning. The key issue is that prediction loss only quantifies the stability of the predictor, saying nothing of the *quality* of the latents that are being predicted.

We address this by training a grid probe (Fig. 1) concurrently with our model. The probe is a linear layer which, given a latent, must predict the occupancy mask of the Tetris grid in that state. We report the final validation F1 score of this probe as "Probe F1", and find that it is a good predictor of downstream latent planning performance.

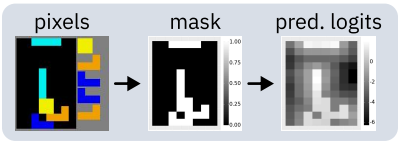


Figure 1. Latent grid probe diagram.

3.4. Patch projector

Our key observation is that existing JEPA world models make opposite and equally unsatisfying choices about how ViT tokens enter the latent state. For an observation, the encoder produces a class token $\mathbf{t}_t^{\text{cls}} \in \mathbb{R}^{D_{\text{tok}}}$ and N patch tokens $\mathbf{t}_t^i \in \mathbb{R}^{D_{\text{tok}}}$. LeWM [16] forms its latent from the class token alone and discards the patch tokens entirely, which costs it the spatial detail needed to track individual cells on a Tetris board. DINO-WM [21] takes the other extreme and stacks all patch tokens, yielding a latent of dimension $N \cdot D_{\text{tok}} = 31\,104$; this preserves spatial detail but makes each planning rollout 150× slower (Tab. 1).

We interpolate between these extremes using a *patch projector* (Fig. 2a): a single linear layer $\mathbf{W}_p \in \mathbb{R}^{D_{\text{proj}} \times D_{\text{tok}}}$, shared across all N patches, that compresses each patch token before it is concatenated with the class token and passed through the encoder's existing output projection $\mathbf{W}_e$:

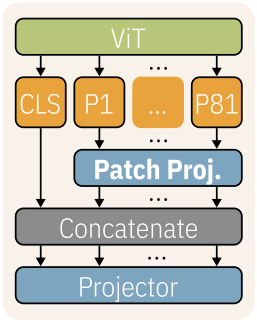
$$\mathbf{z}_t = \mathbf{W}_e \left[\mathbf{t}_t^{\text{cls}} \; ; \; \mathbf{W}_p \mathbf{t}_t^1 \; ; \; \ldots \; ; \; \mathbf{W}_p \mathbf{t}_t^N \right]. \tag{3}$$

Because $\mathbf{W}_p$ is shared, it adds only $D_{\text{proj}} \cdot D_{\text{tok}}$ parameters irrespective of the number of patches, and the intermediate representation has dimension $D_{\text{tok}} + N \cdot D_{\text{proj}}$ rather than $N \cdot D_{\text{tok}}$.

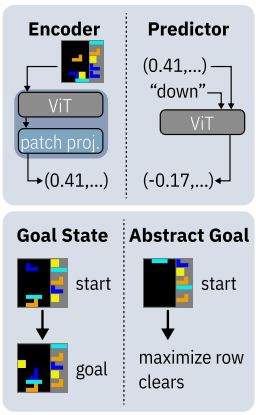
Eq. 3 makes both baselines special cases of a single design axis: $D_{\text{proj}} = 0$ recovers LeWM, and $D_{\text{proj}} = D_{\text{tok}}$ with $\mathbf{W}_p = \mathbf{I}$ recovers DINO-WM. We sweep D_{proj} along this axis in Sec. 4.3.

3.5. Planning in latent space

Goal-state planning. Given a start observation $\mathbf{im}_0$ and a goal observation $\mathbf{im}_g$, we search for an action sequence



(a) Patch projector-enhanced encoder architecture.



(b) Diagram of our method. Top: world model architecture. Bottom: latent planning.

whose predicted terminal latent is closest to $\mathrm{enc}_\theta(\mathbf{im}_g)$. In Tetris, we consider success a one-to-one match of the current grid with the goal. We use a beam search solver with a beam width of 200, a planning horizon of 5 steps and a receding horizon of 1, replanning after every executed action; goal states are sampled 5 environment steps ahead of the start state. At each expansion we score every candidate sequence by $\|\hat{\mathbf{z}}_H - \mathrm{enc}_\theta(\mathbf{im}_g)\|_2$ and retain the 200 lowest-scoring sequences. On PushT, Reacher and TwoRoom we adopt the planning setup of LeWM [16] unchanged.

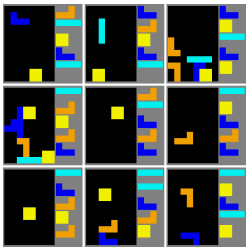
Abstract-goal planning. Goal-state planning presupposes that a target observation is available, which is not the case for the objective a Tetris player actually cares about: clearing as many rows as possible. We therefore replace the goal-matching objective with a learned scalar scorer. A linear layer $\mathbf{w} \in \mathbb{R}^{D_{\text{emb}}}$ maps a latent state to a score $s(\mathbf{z}) = \mathbf{w}^\top \mathbf{z}$, which beam search then minimizes over the planning horizon; lower scores are trained to correspond to board configurations from which more rows are eventually cleared. Since row clears are a sparse, non-differentiable reward, we optimize $\mathbf{w}$ with CMA-ES [12], using the mean number of rows cleared over K episodes as the fitness function.

4. Results

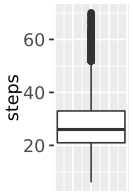
4.1. Setup

A dataset of 25,000 trajectories was generated by playing the game with a strategy that uniformly sampled one of the six available actions (Fig. 3). Play continued until the game over state was reached.

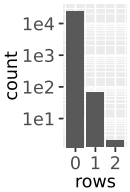
Across all experiments, the train/validation split was 90%. Data was split per-trajectory, not per-window of consecutive steps. For evaluation, we generate a completely separate dataset, using the same base configuration, except for a different seed.



(a) Sample steps from the dataset.



(b) Box plot of steps survived in-game.



(c) Histogram of cleared rows number.

Figure 3. Overview of the generated Tetris dataset.

The model is trained for 10 epochs, under the AdamW optimizer, with a learning rate of 1e-3 and weight decay of 1e-3. A learning rate schedule of linear warmup followed by cosine annealing is used.

4.2. Latent Planning vs. Baselines

We compare the latent planning performance of our world model architecture with baselines (Tab. 1). Our approach[1] comes second best in all the tracked metrics, striking a good balance with a success rate of 92% and a duration of 0.80 seconds per rollout.

Arch.	Seconds /Rollout	Probe F1	Success Rate (%)
PLDM-WM	**0.13**	0.739	24.0
DINO-WM	129.88	**0.974**	**94.5**
LeWM	0.82	0.513	7.0
Ours	<u>0.80</u>	<u>0.949</u>	<u>92.0</u>

Table 1. Comparison of metrics obtained on the Tetris latent planning task, between baseline methods and ours.

4.3. Patch Projector Dimensionality

We ablate over different patch projector dimensionalities to find an optimal value (Tab. 2). When the dimension is set to zero, the architecture is equivalent to plain LeWM. We obtain the best probe F1 and success rate when the dimension is set to 8.

4.4. SIGReg Weight & Post-Training

Fixing the patch projector dimensionality to 8, we ablate over SIGReg weight λ (Tab. 3). Surprisingly, we obtain the best metrics with a weight of $\lambda = 1$ (see Sec. 6 for more).

Higher SIGReg weight leads to worse prediction loss, which is undesirable for tasks such as latent planning. To alleviate this, we introduce a post-training step, denoted with an asterisk. We initialize the encoder with the final

[1]We use a patch projector-enhanced ViT encoder, with dimension set to 8, trained with SIGReg weight of $\lambda = 1$, and post-trained for 50 epochs (see Sec. 4.4).

Patch Proj. Dim.	Pred. Loss	Probe F1	Success Rate (%)
0	0.0314	0.513	7.0
1	0.0374	0.905	85.0
2	0.0319	0.908	88.0
4	0.0293	0.920	90.0
8	0.0304	**0.949**	**90.5**
16	**0.0287**	0.931	86.5

Table 2. Ablation over patch projector dimension.

weights from training with $\lambda = 1$, freeze it, and train only the predictor for 50 more epochs. This results in a noticeable prediction loss improvement, and a slightly higher success rate.

λ	Pred. Loss	Probe F1	Success Rate (%)
0.05	**0.0030**	0.670	39.0
0.1	0.0072	0.789	54.0
1.0	0.0304	**0.949**	90.5
*1.0	0.0213	**0.949**	**92.0**
1.25	0.0321	0.921	88.0
1.6	0.0377	0.911	85.0

Table 3. Ablation over SIGReg weight, along with post-training experiment denoted with an asterisk.

4.5. Baseline Environments

We study the effect of applying our patch projector-enhanced architecture to baseline environments from the stable-worldmodel [15] framework (Fig. 4). The patch projector architecture achieves a slight improvement for pusht, an almost negligible improvement for reacher, and worse performance for tworoom.

Train Dataset	Patch Proj.	Success Rate (%)
pusht	no	2.30 ± 0.53
pusht	yes	**3.80 ± 0.58**
reacher	no	13.20 ± 1.21
reacher	yes	**13.70 ± 1.13**
tworoom	no	**37.70 ± 1.62**
tworoom	yes	32.00 ± 1.41

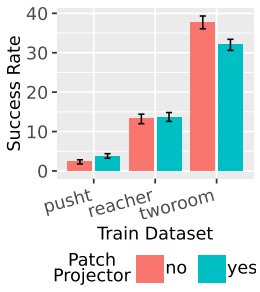


Figure 4. Ablation over LeWM with/without patch projector in baseline environments. The ± denotes standard error of the mean.

4.6. Playing Tetris

Our Tetris-playing setup is able to solve and play forever environments consisting of the $\{I\}$ tetromino set trained with $K = 1$ evaluations, and the $\{I, O\}$ set trained with $K = 5$ evaluations. Once the number of tetrominoes is further increased, the approach breaks down.

5. Limitations

Although adding a patch projector resulted in a dramatic improvement in the Tetris environment, its effect on the baseline environments was significantly less pronounced. We think this could be an issue of improperly tuned hyperparameters (learning rate and SIGReg weight), however we did not search for better values.

We only manage to successfully play a reduced version of Tetris. The most likely culprit is the simplicity of our setup: a single linear layer may not possess sufficient capacity to accurately score the latent space, and CMA-ES would not perform as well if we were to increase the parameter count.

Another factor which likely harms Tetris playing performance is the small amount of row clears that are actually present in the dataset (71 in total), caused by the simplistic strategy we use to gather playthroughs of the game.

6. Discussion

We believe that introducing the patch projector results in such a dramatic improvement in the Tetris environment (compared to the baseline environments), because the Tetris environment consists almost entirely of spatial structure i.e. tiles arranged in a grid. In a similar vein, ViT-based segmentation architectures rely on patch tokens, not on the `[CLS]` token.

An unexpected consequence of our architectural change was improved resistance of the model to being trained under a high SIGReg weight. In LeWorldModel[16] the optimal value is reported to be around $\lambda = 0.1$, with performance degrading sharply when $\lambda \geq 0.5$. Conversely, our architecture achieves the best metrics at $\lambda = 1$, far beyond this threshold. This is a promising discovery, especially if this property also transfers to other environments.

7. Conclusion

We introduced the patch projector, a single shared linear layer that compresses ViT patch tokens before they enter a world model's latent state, resolving the trade-off between LeWM's class-token-only latent and DINO-WM's stacked-token latent. On Tetris, a dense discrete benchmark we developed, it raises planning success from 7% to 92% at unchanged rollout cost. With a CMA-ES-optimised linear scorer, the model plays reduced Tetris indefinitely from row clears alone.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. 1

[2] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. 1, 2

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022. 1

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. 1

[5] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster, 2023. 2

[6] Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments, 2024. 2

[7] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers, 2024. 2

[8] Jingtao Ding, Yunke Zhang, Yu Shang, Yuheng Zhang, Zefang Zong, Jie Feng, Yuan Yuan, Hongyuan Su, Nian Li, Nicholas Sukiennik, et al. Understanding world or predicting future? a comprehensive survey of world models. *ACM Computing Surveys*, 58(3):1–38, 2025. 1

[9] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. *Advances in neural information processing systems*, 31, 2018. 1

[10] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels, 2019. 1

[11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024. 1

[12] Nikolaus Hansen. The cma evolution strategy: A tutorial, 2023. 1, 3

[13] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention, 2021. 2

[14] Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1): 1–62, 2022. 1

[15] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026. 4

[16] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 3, 4

[17] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models, 2023. 1, 2

[18] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2024. 1

[19] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification, 2021. 2

[20] Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim GJ Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *Advances in Neural Information Processing Systems*, 38:43905–43941, 2026. 1

[21] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024. 1, 2